UNIT I: Principles of Object Oriented Programming & Control Structures

Section A: Debugging Questions

Question 1: Operator Precedence Trap with Bitwise Operations & Implicit Promotion

Topic: Operators in C++, Operator Precedence, Implicit Conversion

Corrected Code:

```
#include <iostream>
using namespace std;

int main() {
   unsigned char num = 15;

   if (((num & 1) == 1) && (num > 10)) {
      cout << "Valid Odd Number: "
           << (int)num << endl;
   }
   else {
      cout << "Condition Failed for: "
           << (int)num << endl;
   }

   for (int i = 3; i >= 0; --i) {
      cout << i << " ";
   }

   cout << endl;
   return 0;
}
```

Output:

```
Valid Odd Number: 15
3 2 1 0
```

Question 2: Scope Resolution & Variable Shadowing Conflicts

Topic: Variables, Scope Resolution Operator (`::`), Constants

Corrected Code:

```
#include <iostream>
#include <iomanip>
using namespace std;

double taxRate = 0.18;

int main() {
   double itemPrice = 1000.0;
   double subsidizedRate = 0.05;

   cout << fixed << setprecision(2);

   cout << "Local Rate: "
        << subsidizedRate << endl;

   double totalWithGlobalTax =
      itemPrice + (itemPrice * ::taxRate);

   {
      int corporateRate = 12;

      double totalWithLocalTax =
         itemPrice + (itemPrice * subsidizedRate);

      cout << "Inner total: "
           << totalWithLocalTax << endl;

      cout << "Corporate Rate: "
           << corporateRate << "%" << endl;
   }

   cout << "Final Global Calculation: "
        << totalWithGlobalTax << endl;

   return 0;
}
```

Output: Local Rate: 0.05

Inner total: 1050.00

Corporate Rate: 12%

Final Global Calculation: 1180.00

Question 3: Switch-Case Variable Initialization & Illegal Case Types

Topic: Control Structures (`switch-case`), Tokens, Data Types

Corrected Code:

```
#include <iostream>
using namespace std;

int main() {
    int tokenType = 2;
    float tokenValue = 0.0f;

    switch (tokenType) {

        case 1: {
            int offset = 10;
            tokenValue = 50.0f + offset;
            break;
        }

        case 2: {
            int multiplier = 5;
            tokenValue = 20.0f * multiplier;
            break;
        }

        case 3: {
            tokenValue += 100.0f;
            break;
        }

        case 5: {
            cout << "Float token detected" << endl;
            break;
        }

        default:
```

```
        cout << "Unknown token" << endl;
    }

    cout << "Calculated Value: "
         << tokenValue << endl;

    return 0;
}
```

Output: Calculated Value: 100

Question 4: Reference Variable Initialization & Constant Binding Violations

Topic: Reference Variables, Constants, Implicit Conversions

Corrected Code:

```
#include <iostream>
using namespace std;

int main() {
    int a = 25;

    int &refA = a;

    const int &refLiteral = 100;

    double pi = 3.14159;

    int convertedPi = static_cast<int>(pi);
    int &refInt = convertedPi;

    const int &constRef = 50;

    cout << "a and refA: "
         << a << " " << refA << endl;

    cout << "refLiteral: "
         << refLiteral << endl;

    cout << "refInt: "
         << refInt << endl;
```

```
    cout << "constRef: "
         << constRef << endl;

    return 0;
}
```

Output:

```
a and refA: 25 25
refLiteral: 100
refInt: 3
constRef: 50
```

Question 5: Dynamic Array Allocation & Stray Loop Semicolon

Topic: Memory Operators (`new`, `delete`), Loops, Off-by-One Boundaries

Corrected Code:

```
#include <iostream>
using namespace std;

int main() {
    const int size = 5;

    int *readings = new int[size];

    for (int i = 0; i < size; ++i) {
        readings[i] = (i + 1) * 10;
    }

    int total = 0;

    for (int i = 0; i < size; ++i) {
        total += readings[i];
    }

    cout << "Readings: ";

    for (int i = 0; i < size; ++i) {
        cout << readings[i] << " ";
    }
```

```
    cout << endl;

    cout << "Total: " << total << endl;

    delete[] readings;

    return 0;
}
```

Output:

Readings: 10 20 30 40 50

Total: 150

Section B: Pseudo-Code Writing Questions

Question 1: Procedural-to-OOP Paradigm Transformation

Topic: OOP Paradigm, Encapsulation, Data Hiding, Abstraction

pseudo-code:

```
CLASS WaterConsumer
PRIVATE:
    consumerID : STRING
    previousReading : REAL
    currentReading : REAL
    unitsConsumed : REAL
    totalBill : REAL
    validReading : BOOLEAN

PUBLIC:
    CONSTRUCTOR Initialize(id : STRING, prev : REAL, curr : REAL)
        consumerID ← id
        IF prev < 0 OR curr < 0 OR curr < prev THEN
            validReading ← FALSE
            DISPLAY "ERROR: Invalid meter readings"
        ELSE
            previousReading ← prev
            currentReading ← curr
            validReading ← TRUE
            unitsConsumed ← 0
            totalBill ← 0
        ENDIF
    END CONSTRUCTOR

    METHOD CalculateConsumption()
        IF validReading = TRUE THEN
            unitsConsumed ← currentReading - previousReading
        ENDIF
    END METHOD

    METHOD ComputeBill()
        DECLARE firstSlab, secondSlab, thirdSlab, surcharge : REAL
        IF validReading = FALSE THEN
            RETURN
        ENDIF
```

```
        firstSlab ← MIN(unitsConsumed, 100) * 1.50
        secondSlab ← MIN(MAX(unitsConsumed - 100, 0), 200) * 2.50
        thirdSlab ← MAX(unitsConsumed - 300, 0) * 4.00

        totalBill ← firstSlab + secondSlab + thirdSlab
        surcharge ← 0

        IF unitsConsumed > 300 THEN
            surcharge ← totalBill * 0.10
        ENDIF
        totalBill ← totalBill + surcharge
    END METHOD

    METHOD DisplayBill()
        IF validReading = FALSE THEN
            DISPLAY "Invoice cannot be generated"
        ELSE
            DISPLAY "Consumer ID: ", consumerID
            DISPLAY "Units Consumed: ", unitsConsumed
            DISPLAY "Energy/Water Charge: ", totalBill
            DISPLAY "Total Payable: ", totalBill
        ENDIF
    END METHOD
ENDCLASS

BEGIN MAIN
    DECLARE consumer : WaterConsumer
    consumer.Initialize("WC101", 1450.0, 1820.0)
    consumer.CalculateConsumption()
    consumer.ComputeBill()
    consumer.DisplayBill()
END MAIN
```

output:

```
Consumer ID: WC101
Units Consumed: 370
Energy/Water Charge: 1023
Total Payable: 1023
```

Question 2: Multi-Tier Tax & Precision Cast Engine

Topic: Control Structures (Selection), Data Types, Type Casting

pseudo-code:

```
CLASS TaxCalculationEngine
PRIVATE:
    grossAnnualSalary : REAL
    deduction80C : REAL
    deductionHealthInsurance : REAL
    eligible80C : REAL
    eligibleHealth : REAL
    taxableIncome : REAL
    rawTax : REAL
    cess : REAL
    finalTax : REAL

PUBLIC:
    METHOD Calculate(gross : REAL, d80 : REAL, health : REAL)
        grossAnnualSalary ← gross
        eligible80C ← MIN(d80, 150000)
        eligibleHealth ← MIN(health, 25000)

        taxableIncome ← MAX(grossAnnualSalary - eligible80C - eligibleHealth, 0)
        rawTax ← 0

        IF taxableIncome > 250000 THEN
```

```
            rawTax ← rawTax + MIN(taxableIncome - 250000, 250000) * 0.05
        ENDIF

        IF taxableIncome > 500000 THEN
            rawTax ← rawTax + MIN(taxableIncome - 500000, 500000) * 0.20
        ENDIF

        IF taxableIncome > 1000000 THEN
            rawTax ← rawTax + (taxableIncome - 1000000) * 0.30
        ENDIF

        cess ← rawTax * 0.04
        finalTax ← INTEGER(rawTax + cess + 0.5)
    END METHOD

    METHOD Display()
        DISPLAY "Eligible 80C: ", eligible80C
        DISPLAY "Eligible Health Insurance: ", eligibleHealth
        DISPLAY "Taxable Income: ", taxableIncome
        DISPLAY "Raw Tax: ", rawTax
        DISPLAY "Health and Education Cess: ", cess
        DISPLAY "Final Tax: ", finalTax
    END METHOD
ENDCLASS

BEGIN MAIN
    DECLARE engine : TaxCalculationEngine
    engine.Calculate(1200000, 180000, 30000)
    engine.Display()
END MAIN
```

output:

```
Eligible 80C: 150000
Eligible Health Insurance: 25000
Taxable Income: 1025000
Raw Tax: 120000
Health and Education Cess: 4800
Final Tax: 124800
```

Question 3: Bitwise Mask Validation & Sensor Health Telemetry

pseudo-code:

```
CONSTANT GYRO_MASK : BYTE ← 0x01
CONSTANT ACCEL_MASK : BYTE ← 0x02
CONSTANT BARO_MASK : BYTE ← 0x04
CONSTANT GPS_MASK : BYTE ← 0x08
CONSTANT MANDATORY_MASK : BYTE ← 0x0F
CONSTANT LOW_BATTERY_MASK : BYTE ← 0x10
CONSTANT OVER_TEMP_MASK : BYTE ← 0x20
CONSTANT RADIO_MASK : BYTE ← 0x40
CONSTANT FATAL_MASK : BYTE ← 0x80
CONSTANT INHIBITOR_MASK : BYTE ← 0xB0

METHOD CheckFlightReadiness(statusRegister : BYTE)
    DECLARE missingMask, activeInhibitors : BYTE
    DECLARE ready : BOOLEAN
    ready ← TRUE

    missingMask ← (statusRegister AND MANDATORY_MASK) XOR MANDATORY_MASK
    activeInhibitors ← statusRegister AND INHIBITOR_MASK

    IF (statusRegister AND LOW_BATTERY_MASK) <> 0 THEN
        DISPLAY "ERROR: Low battery warning (bit 4)"
        ready ← FALSE
    ENDIF
    IF (statusRegister AND OVER_TEMP_MASK) <> 0 THEN
```

```
        DISPLAY "ERROR: Over-temperature warning (bit 5)"
        ready ← FALSE
    ENDIF
    IF (statusRegister AND FATAL_MASK) <> 0 THEN
        DISPLAY "ERROR: Fatal actuator failure (bit 7)"
        ready ← FALSE
    ENDIF

    IF missingMask <> 0 THEN
        DISPLAY "ERROR: Missing mandatory sensor mask = ", HEX(missingMask)
        IF (missingMask AND GYRO_MASK) <> 0 THEN DISPLAY "Gyroscope missing" ENDIF
        IF (missingMask AND ACCEL_MASK) <> 0 THEN DISPLAY "Accelerometer missing" ENDIF
        IF (missingMask AND BARO_MASK) <> 0 THEN DISPLAY "Barometric altimeter missing" ENDIF
        IF (missingMask AND GPS_MASK) <> 0 THEN DISPLAY "GPS fix missing" ENDIF
        ready ← FALSE
    ENDIF

    IF ready = TRUE THEN
        DISPLAY "FLIGHT READY"
    ELSE
        DISPLAY "FLIGHT NOT READY"
    ENDIF
END METHOD
```

output:

```
ERROR: Low battery warning (bit 4)
ERROR: Over-temperature warning (bit 5)
ERROR: Missing mandatory sensor mask = 0x04
Barometric altimeter missing
FLIGHT NOT READY
```

Question 4: Optimized Prime Factorization & Divisor Engine

Topic: Control Structures (Iteration: `for`/`while`, Jump: `break`, `continue`, `goto`)

pseudo-code:

```
BEGIN DivisorAndPrimeEngine
    DECLARE N, d, pairedDivisor, sumOfDivisors : INTEGER

    INPUT N
    WHILE N <= 0 DO
        DISPLAY "Invalid input. Enter a positive integer."
        INPUT N
    ENDWHILE

    IF N = 1 THEN
        sumOfDivisors ← 0
    ELSE
        sumOfDivisors ← 1
        FOR d ← 2 TO FLOOR(SQRT(N)) STEP 1
            IF N MOD d <> 0 THEN
                CONTINUE
            ENDIF

            pairedDivisor ← N DIV d
            sumOfDivisors ← sumOfDivisors + d

            IF pairedDivisor <> d THEN
                sumOfDivisors ← sumOfDivisors + pairedDivisor
            ENDIF
        ENDFOR
    ENDIF

    DISPLAY "Sum of proper divisors: ", sumOfDivisors

    IF sumOfDivisors = N THEN
```

```
        DISPLAY "Perfect Number"
    ELSE IF sumOfDivisors > N THEN
        DISPLAY "Abundant Number"
    ELSE
        DISPLAY "Deficient Number"
    ENDIF
END DivisorAndPrimeEngine
```

output:

Trace for N = 28

Result: sum = 28, therefore 28 is a Perfect Number.

```
Sum of proper divisors: 28
Perfect Number
```

Question 5: State-Machine ATM Controller with Sentinel Loops

Topic: Control Structures (`switch-case`, `while`, nested `if-else`), Data Types

pseudo-code:

```
BEGIN ATM_SessionController
    CONSTANT CORRECT_PIN : INTEGER ← 4821
    DECLARE enteredPIN, attempts, option : INTEGER
    DECLARE balance, amount : REAL
    DECLARE authenticated : BOOLEAN

    balance ← 1000.00
    attempts ← 0
    authenticated ← FALSE

    WHILE attempts < 3 AND authenticated = FALSE DO
        DISPLAY "Enter 4-digit PIN:"
        INPUT enteredPIN
        attempts ← attempts + 1

        IF enteredPIN = CORRECT_PIN THEN
            authenticated ← TRUE
        ELSE
            DISPLAY "Incorrect PIN"
        ENDIF
    ENDWHILE

    IF authenticated = FALSE THEN
        DISPLAY "Card locked. Maximum attempts exceeded."
        STOP
    ENDIF

    DISPLAY "Authentication successful"

    option ← 0
    WHILE option <> 4 DO
        DISPLAY "1. Balance Inquiry"
        DISPLAY "2. Cash Deposit"
        DISPLAY "3. Cash Withdrawal"
        DISPLAY "4. Exit"
        INPUT option

        SWITCH option
            CASE 1:
                DISPLAY "Available balance: ", balance
                BREAK

            CASE 2:
                DISPLAY "Enter deposit amount:"
```

```
                INPUT amount
                IF amount > 0 THEN
                    balance ← balance + amount
                    DISPLAY "Deposit successful"
                    DISPLAY "Updated balance: ", balance
                ELSE
                    DISPLAY "Invalid deposit amount"
                ENDIF
                BREAK

            CASE 3:
                DISPLAY "Enter withdrawal amount:"
                INPUT amount
                IF amount <= 0 THEN
                    DISPLAY "Invalid withdrawal amount"
                ELSE IF amount > balance THEN
                    DISPLAY "Insufficient balance"
                ELSE IF amount MOD 100 <> 0 THEN
                    DISPLAY "Rejected: amount must be a multiple of 100"
                ELSE
                    balance ← balance - amount
                    DISPLAY "Withdrawal successful"
                    DISPLAY "Remaining balance: ", balance
                ENDIF
                BREAK

            CASE 4:
                DISPLAY "Thank you. Session ended."
                BREAK

            DEFAULT:
                DISPLAY "Invalid menu option"
        ENDSWITCH
    ENDWHILE
END ATM_SessionController
```

output:

```
Enter 4-digit PIN:
1234
Incorrect PIN
Enter 4-digit PIN:
4821
Authentication successful
1. Balance Inquiry
2. Cash Deposit
3. Cash Withdrawal
4. Exit
3
Enter withdrawal amount:
250
Rejected: amount must be a multiple of 100
1. Balance Inquiry
2. Cash Deposit
3. Cash Withdrawal
4. Exit
3
Enter withdrawal amount:
300
Withdrawal successful
Remaining balance: 700
1. Balance Inquiry
2. Cash Deposit
3. Cash Withdrawal
4. Exit
4
Thank you. Session ended.
```